Web3Geeks Production-Ready Client Onboarding & Proposal Agent

Executive Report — OpenRouter Edition

1. Business Goal

The system automates first-pass client inquiry qualification and proposal drafting for a software/AI services agency. It standardizes incoming requirements, checks budget and timeline feasibility, uses company/service data, produces a structured proposal, and stops before client-facing approval so a human reviewer retains commercial control.

2. Architecture & Technology Choice

FastAPI is the service boundary and Pydantic performs request validation. A LangGraph workflow executes enrichment, deterministic qualification, OpenRouter proposal generation, quality/safety review, and conditional self-correction. The agent reads a company policy file and queries a SQLite service catalogue. Pending proposals are persisted and exposed through a browser approval page and JSON approval endpoint.

LangGraph is preferred over a role-based multi-agent framework because this workflow is control-heavy: the order of operations, state, review gate, revision path, and human checkpoint are explicit. OpenRouter is used as the model gateway through its OpenAI-compatible API, allowing the project to select a model such as **openai/gpt-4.1-mini** without coupling the business workflow to one provider.

3. Reliability, Safety & Human Oversight

Input validation rejects malformed requests and obvious prompt-injection-like instructions before model execution. File/database tools have timeouts and safe fallback state. Model failures are surfaced as failures rather than being silently represented as successful generations. The proposal review checks required fields, commercial consistency, and prohibited guarantee language; an unsuccessful review can route through a revision node.

The human checkpoint is now operational rather than merely conceptual. After **POST /agent/run**, the response contains an approval URL. A reviewer opens **/approvals**, reads the request and proposal, enters notes, and explicitly clicks Approve or Reject. The decision is persisted in SQLite and the JSON approval endpoint remains available for programmatic clients.

4. Evaluation Framework

The evaluation suite contains eight cases: six normal business requests, one low-budget/short-timeline feasibility edge case, and one prompt-injection adversarial case. Six criteria are scored from 1–5: task success, factual/commercial consistency, proposal quality, safety, latency, and token efficiency. The evaluator calls the configured OpenRouter model and writes the actual measurements to **eval/evaluation_results.csv**.

Criterion	What is measured
Task success	Correct workflow outcome and feasibility routing
Factual consistency	Price/timeline respect service catalogue constraints
Quality	Scope, assumptions, exclusions, risks and next steps
Safety	No unsafe guarantees or bypass of safeguards
Latency	End-to-end response time
Efficiency	Provider-reported token usage

5. Known Limitations & Failure Pattern

The main expected edge-case failure is commercial infeasibility: a client may request an AI agent below the service minimum price or timeline. The revised implementation adds a feasibility gate so these cases are explicitly marked **needs_discovery**, and the proposal must surface the constraint rather than treating it as an ordinary feasible lead. A larger historical evaluation set is still required before production claims can be made.

6. Monitoring & Production Readiness

JSON logs capture model name, model latency, end-to-end latency, prompt/completion tokens, estimated cost, tool events, and errors. Recommended alerts include 5xx >2% over 15 minutes, model/tool failures >5%, p95 latency >8 seconds, cost per successful run >1.5× baseline, or offline evaluation below 4/5. Smoke evaluation should run after every model/prompt/workflow change; broader regression should run weekly during active development and monthly after stabilization.

7. Recommended Next Steps

For a production deployment, replace SQLite with managed PostgreSQL, add authentication and role-based access control to the approval interface, connect a CRM/email system, use provider-reported billing data rather than configured rate estimates, add durable graph persistence, and expand evaluation with real historical inquiries plus human reviewer labels.

Submission note: generate the final evaluation CSV after configuring the student's OpenRouter API key. The repository deliberately avoids fabricated model results.